Data-Owner & Steward Framework — Microsoft Purview

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Data-Owner & Steward Framework (Microsoft Purview) 4
- What you get . . . 4
- What this delivers . . . 4
- The deployment model . . . 4
- Scope boundary . . . 5
- Outcome . . . 5

1. Solution Overview — Data-Owner & Steward Framework (Microsoft Purview) 5
- What this solution is . . . 5
- Why this matters . . . 6
- The three building blocks . . . 6
- Roles and least privilege . . . 7
- Pilot-first, safe by design . . . 7
- Licensing and the evolving portal . . . 7
- What this is not . . . 7
- Naming convention (used in every doc and script) . . . 7
- The standard deployment model . . . 8

2. Requirements Guide — Data-Owner & Steward Framework (Microsoft Purview) 8
- 2.1 Purview foundation already stood up (roles, audit, workloads connected) . . . 8
- 2.2 Compliance licensing for the surfaces in scope . . . 8
- 2.3 Least-privilege stewardship roles (compliance side) . . . 9
- 2.4 Unified Catalog governance-domain roles (data-governance side, if in scope) . . . 9
- 2.5 Microsoft Graph toolchain and read scopes . . . 9
- 2.6 The in-scope data assets and control inventory . . . 9
- 2.7 Named stakeholders and the RACI authority . . . 10
- 2.8 A pilot scope and review cadence agreed . . . 10

Global Requirements — Microsoft Purview Solutions 10
- 1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 10
- 2. Purview roles (least-privilege) and portal access . . . 11
- 3. Security & Compliance PowerShell + Graph toolchain . . . 11
- 4. Data sources & scope in scope . . . 11
- 5. Compliance, records, privacy & legal stakeholders . . . 12
- 6. A pilot scope and change control . . . 12

3. Implementation Guide — Data-Owner & Steward Framework (Microsoft Purview) 12
- Step 1 — Assess current ownership coverage (Assess) . . . 13
- Step 2 — Design the role model and RACI (Design) . . . 13

Step 3 — Preview and apply stewardship assignments (Configure, pilot-first) . . . 13
Step 4 — Establish the review cadence (Configure) . . . 14
Step 5 — Validate on the pilot domain (Pilot → Validate) . . . 14
Step 6 — Expand across the estate (Expand) . . . 15
5. Testing Guide — Data-Owner & Steward Framework (Microsoft Purview) 15
Test 1 — Ownership inventory is complete . . . 15
Test 2 — RACI is well-formed . . . 15
Test 3 — Assignment dry-run makes no changes . . . 15
Test 4 — Every pilot control/asset has an owner and a steward . . . 16
Test 5 — Named roles resolve to real identities . . . 16
Test 6 — Separation of duties holds . . . 16
Test 7 — Review cadence produces a correct due/overdue list . . . 16
Test 8 — Accountability report reflects reality . . . 16
Pass/fail gate . . . 16
6. Troubleshooting Guide — Data-Owner & Steward Framework (Microsoft Purview) 17
A capability or surface is missing from the Purview portal . . . 17
`Connect-IPPSSession` fails or cmdlets are "not recognized" . . . 17
`DGSTEWARD-Assess-Ownership.ps1` shows everything as "Unowned" . . . 17
The governance-domain rows are skipped . . . 17
`DGSTEWARD-Set-Stewardship.ps1` "changed nothing" . . . 17
A role group named in the matrix "does not exist" . . . 18
`Set-Label` / `Set-DlpCompliancePolicy` reports access denied on `-Commit` . . . 18
Named owners/stewards don't resolve in the accountability report . . . 18
Cadence check shows everything "NEVER-REVIEWED" . . . 18
"Who is allowed to hold the Insider Risk / eDiscovery role?" . . . 18
The framework "passed" but ownership drifted a quarter later . . . 19
7. Best Practices — Data-Owner & Steward Framework (Microsoft Purview) 19
8. Deliverables Summary — Data-Owner & Steward Framework (Microsoft Purview) 20
Documentation . . . 20
PowerShell scripts . . . 21
What the buyer receives . . . 21
Outcome . . . 22
Appendix A — PowerShell Scripts 23

Executive Summary — Data-Owner & Steward Framework (Microsoft Purview)

Data assets accumulate faster than accountability for them does. Sites, sensitivity labels, DLP policies, retention rules, and governance domains all exist in Microsoft Purview — but ask "who owns this?" and the honest answer is usually "nobody in particular." Unowned data is both a **compliance gap** (no one re-certifies its classification or answers for it in an audit) and a **data-risk exposure** (a control sits in test mode for a year and no one notices). This solution closes that gap: it stands up a **data-owner and data-steward operating framework** — **roles, a RACI, and a review cadence** — that assigns named, defensible accountability to your data and to the Purview controls that govern it.

It gives you a repeatable way to define the stewardship **roles** (Data Owner, Data Steward, Governance Domain Owner, Control Custodian, with sensitive roles kept strictly separate), record responsibility in a **RACI** (exactly one Accountable party per governance activity), and keep it all current on a **review cadence** — using only the Microsoft Purview portal, Security & Compliance PowerShell, and Microsoft Graph.

This is a complete, deploy-it-today package for the accountability operating model, written for a competent M365/compliance administrator who already runs Purview and needs ownership assigned across it.

What you get

• A plain-language Solution Overview of the roles, the RACI, the review cadence, and the least-privilege model.

• A Requirements Guide with verify / implement-if-missing for every prerequisite, including which surfaces your licensing actually covers.

• An Implementation Guide of precise, ordered portal + PowerShell + Graph steps — assess coverage, design the RACI, apply assignments pilot-first, establish the cadence — with every control/asset named and pass/fail gates plus rollback.

• Five PowerShell scripts: ownership/coverage discovery, a stewardship-assignment helper (`-WhatIf` by default), a framework validator, an accountability/coverage report, and a review-cadence health helper.

• Testing and Troubleshooting guides for a confident, safe rollout.

What this delivers

Every in-scope data asset and governance control with a named **Data Owner** and **Data Steward**, responsibility recorded in a well-formed RACI, sensitive roles named and separated, and a review cadence that keeps ownership from decaying — validated on a bounded pilot domain before it is rolled across the estate.

The deployment model

Assess → Design → Configure → Pilot → Validate → Expand.

Scope boundary

This is the **accountability operating model**, not the technical controls. Building the label taxonomy, authoring DLP and retention policies, and rolling out the Data Map / scanning are each the subject of a dedicated Purview solution that this framework complements. And it records decisions rather than making them: *what* an asset is classified as, *what* its retention or records schedule is, *whether* a regulation applies, and *who may* hold a sensitive role are determinations deferred to the organization's compliance, records, privacy, and legal functions — this solution provides the method to operate those decisions with named accountability and evidence, not the determination.

Outcome

Accountability assigned across your Microsoft Purview estate — every asset and control owned, responsibility unambiguous in a RACI, and the framework kept alive by a review cadence — configured, piloted, validated, and defensible.

1. Solution Overview — Data-Owner & Steward Framework (Microsoft Purview)

What this solution is

Data assets accumulate faster than accountability for them does. A SharePoint site, a sensitivity label, a DLP policy, a governance domain in the Purview Unified Catalog, a set of retention rules — all exist, but ask "who *owns* this?" and the honest answer is usually "nobody in particular." This solution delivers the missing layer: a **data-owner and data-steward operating framework** inside Microsoft Purview. It assigns **named accountability** to data assets and to the Purview objects that govern them, records that accountability in a **RACI** (who is Responsible, Accountable, Consulted, Informed for each governance activity), and puts it on a **review cadence** so ownership stays current instead of decaying the day the project ends.

It is the *operating model* — roles, RACI, and cadence — not a new technical control. It sits **on top of** the Purview capabilities you already run (sensitivity labels, DLP, retention/records, the Unified Catalog / Data Map, insider risk, communication compliance) and gives each of them an accountable owner and a steward who keeps it healthy. Everything is done with the **Microsoft Purview portal** (purview.microsoft.com), **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** — no third-party tooling.

> ***Boundary.*** *This solution provides the technical method to *define stewardship roles, assign named owners and stewards, record a RACI, and run a review cadence* in Microsoft Purview. It does **not** make the business, regulatory, records, or legal **determinations** — *what a data asset is worth*, *what its retention or records schedule is*, *which classification applies*, or *who is permitted to hold a sensitive role*. Those belong to the organization's data owners, compliance, records, privacy, and legal functions. The framework gives them a governed structure to *record and operate* those decisions; it never invents the decision.*

Why this matters

- **Unowned data is an accountability gap and a risk exposure.** When no one owns a data asset, no one

re-certifies its classification, notices its DLP policy is in test mode a year later, or answers for it in an audit. "Who signed off on this?" has no answer.

- **Ownership without a RACI is ambiguous.** Two people each assuming the other owns re-certification is

the same as nobody owning it. A RACI removes the ambiguity: exactly one **Accountable** party per activity, named **Responsible** doers, and explicit **Consulted / Informed** parties.

- **Ownership without a cadence decays.** A framework defined once and never revisited is stale within a

quarter. The review cadence — re-certify owned assets, chase unowned gaps, retire what's obsolete — is what keeps the framework *alive*, and it is a first-class deliverable here, evidenced by the scripts.

- **Purview objects need owners too.** Labels, DLP policies, retention policies, and governance domains

are governance *controls*; a control with no owner drifts. This framework names an owner and steward for the controls as well as the data.

The three building blocks

This solution stands up three things, each named once in the Implementation Guide and reused everywhere.

1. **Roles** — the named stewardship roles and who fills them:

- **Data Owner** — accountable for a data asset or a governance domain and for the decisions about it

(classification intent, retention intent, access intent). Business-side, senior, few in number.

- **Data Steward** — curates the asset day to day: keeps classification/labels, ownership metadata,

and status current, runs the review cadence, and surfaces gaps. The operational role.

- **Governance Domain Owner** — owns a governance domain in the Purview Unified Catalog and its role

assignments (where the Data Governance capability is in use).

- **Data Custodian / Control Owner** — accountable for a specific Purview *control* (a label, a DLP

policy, a retention policy) — often IT/compliance — distinct from the *data* owner.

- Sensitive-data roles (**Insider Risk**, **Communication Compliance**, **eDiscovery**) are named and

kept strictly separate; who may hold them is an HR/privacy/legal determination.

1. **RACI** — for each recurring governance activity (classify/label an asset, author or tune a DLP or

retention policy, re-certify ownership, review a disposition, respond to an access request, close an audit finding), the RACI names exactly one **Accountable** party and the **Responsible / Consulted / Informed**

parties. The RACI is the contract that turns roles into unambiguous responsibility.

1. **Review cadence** — the recurring rhythm the steward runs: re-certify owned assets on a schedule, escalate unowned or overdue assets to the Data Owner, retire obsolete assignments, and produce the accountability report. Cadence is what prevents decay.

Roles and least privilege

Where the framework touches Purview permissions, it uses **least-privilege built-in roles**, assigned at the narrowest scope that works. Day-to-day compliance stewardship uses S&C role groups (e.g. **Compliance Administrator**, **Information Protection Admin**, **Records Management**); Unified Catalog stewardship uses **governance-domain-level** roles (**governance domain owner**, **data owner**, **data steward**, **data reader**). Account-level administration is reserved for a minimal named group. Least-privilege assignment is an explicit step in the Implementation Guide, not an afterthought — each role is named at the stage it is needed.

Pilot-first, safe by design

Defining a framework is low-blast-radius (recording that Jane owns a domain changes no user data), but the same discipline applies: **pilot on a bounded scope first** — one or two governance domains / a small named asset set — validate the RACI and cadence there, and only then **expand** to the full estate. Any role assignment the scripts perform is `-WhatIf` **by default**, gated behind an explicit `-Commit`, and prints the exact reverse command. **No secret is ever stored in a script or document.** This mirrors the category model: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand.**

Licensing and the evolving portal

The compliance-side surfaces this framework operates over (labels, DLP, retention, role groups) require the corresponding **Microsoft 365 / E5 / E5 Compliance** licensing per capability. The **Unified Catalog / Data Governance** side (governance domains, data products, governance-domain roles) is billed through a **consumption / pay-as-you-go** model tied to an **Azure subscription** and is **actively evolving** — feature names, portal paths, and data-plane API versions may have moved or may be in preview. **Verify current availability, pricing, portal paths, and API versions for your own tenant** in Microsoft's documentation and in the Purview portal before you plan. This solution cites the method; it never asserts a price, a portal path, or an API version as permanent, and it does not invent cmdlets.

What this is not

This is the *accountability operating model*. It is **not** the label taxonomy design, the DLP/retention policy build, the full Data Map / scanning rollout, or the business-glossary term-curation program — those are separate Purview solutions this framework complements. A steward here may operate a glossary or a set of labels, but *building* them is out of scope. This solution focuses squarely on **who is accountable, for what, and how often it is reviewed.**

Naming convention (used in every doc and script)

Artifacts follow `<Purpose> – Purview – <Stage>` and are reused verbatim across docs, scripts, and portal.

Artifact	Name
Ownership / coverage assessment export	`Data Ownership Inventory – Steward – Assess`
The RACI matrix (roles × activities)	`Data-Owner/Steward RACI – Design`
Applied stewardship role assignments	`Stewardship Assignments – Configure`
Accountability / coverage report	`Accountability Coverage – Report`
Review-cadence health snapshot	`Review Cadence – Health`
Pilot domain / bounded scope	`Stewardship Pilot Domain – Stage1`

The standard deployment model

> *Assess current ownership coverage and gaps → design the role model and RACI → configure the role assignments and cadence → pilot on a bounded domain/asset set → validate every gate → expand across the estate. Pattern:* ***Assess → Design → Configure → Pilot → Validate → Expand.***

2. Requirements Guide — Data-Owner & Steward Framework (Microsoft Purview)

Verify **every** requirement before configuring. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites are in `global-requirements-purview.md` (the six shared requirements: compliance licensing, least-privilege Purview roles, the S&C PowerShell + Graph toolchain, in-scope data sources, compliance/records/privacy/legal stakeholders, and a pilot scope with change control) — those are referenced here, not repeated.

2.1 Purview foundation already stood up (roles, audit, workloads connected)

Description: A working Purview foundation — the portal organized, least-privilege role groups in place, the unified audit log enabled, and the in-scope M365 workloads connected. This framework operates *on top of* that foundation. **Why:** You cannot assign accountability for controls that aren't connected or visible; the audit log is what evidences the cadence. (Extends Global Requirements 2, 4.) **Verify:** In the Purview portal the capabilities in scope (Information Protection, DLP, Data Lifecycle, Unified Catalog) appear and render; role groups resolve. Run `DGSTEWARD-Assess-Ownership.ps1` — it reports connectivity and the objects it can see. **Implement:** If the foundation is not in place, stand it up first (the *Implementing Microsoft Purview: Foundation, Roles & Onboarding* solution) before running this framework.

2.2 Compliance licensing for the surfaces in scope

Description: The **compliance licensing** each in-scope surface requires — E3 for baseline label/DLP, **Microsoft 365 E5 / E5 Compliance** for advanced DLP, auto-labeling, records management, and Audit Premium; and, for the Unified Catalog / Data Governance side, an **Azure subscription** for the **consumption-based** Data Governance billing. **Why:** Purview capabilities are plan-gated; the framework must record which surfaces are actually available so ownership is assigned only to controls you can operate (Global Requirement 1). **Verify:** M365 admin center → `Billing → Licenses`; confirm each

capability appears in the Purview portal. For Data Governance, confirm the linked Azure subscription and that consumption billing is understood. **Verify against your own tenant and plan — capability/pricing availability evolves. Implement:** Assign/purchase the required licensing; where a surface is unlicensed, record it as out-of-scope for ownership assignment for now rather than attempting to enable it.

2.3 Least-privilege stewardship roles (compliance side)

Description: Access to assign members to the built-in S&C role groups the framework uses — **Compliance Administrator** (to operate), **Information Protection Admin**, **Records Management**, and the visibility roles (**Content Explorer Content Viewer / List Viewer**) — with Global Administrator reserved for the rare tenant-wide action. Per Global Requirement 2. **Why:** Assigning stewardship means adding named people to the correct least-privilege role group; over- granting Global Administrator undermines the accountability model you are building. **Verify:** Purview portal → `Settings → Roles & scopes → Role groups` (verify path in your tenant); `Connect-IPPSSession` succeeds and `Get-RoleGroup` returns the groups. **Implement:** Add the operating account to **Compliance Administrator**; reserve Global Administrator for one-time actions only.

2.4 Unified Catalog governance-domain roles (data-governance side, if in scope)

Description: Where the **Unified Catalog / Data Governance** is in scope, the governance-domain-level roles — **governance domain owner**, **data owner**, **data steward**, **data reader** — assigned at the **domain** level, plus a minimal account-level admin group. **Why:** Catalog stewardship is scoped by governance domain; assigning owners/stewards there is how accountability is recorded for catalogued assets. **Verify:** Purview portal → Unified Catalog → the governance domain → its roles (verify the exact path and role names in your tenant — the Data Governance experience is evolving and partly in preview). **Implement:** Create the governance domain(s) and assign the domain roles to named people. The *determination* of who owns a domain is a business decision; you implement the assignment.

2.5 Microsoft Graph toolchain and read scopes

Description: The **Microsoft Graph PowerShell SDK** with a modern-auth/MFA admin, and the delegated read scopes the assessment/report scripts use: `Organization.Read.All`, `Directory.Read.All`, `User.Read.All`, `Group.Read.All` (to resolve named owners/stewards to real identities). No write scope is required for the read-only scripts. Per Global Requirement 3. **Why:** The assessment, accountability, and cadence reports read licensing, group, and (where available) governance-domain data through Graph; keeping to read scopes matches the safe-by-default posture. **Verify:** `Connect-MgGraph` succeeds with the scopes; `DGSTEWARD-Report-Accountability.ps1` (read-only) reports if consent is missing. **Implement:** `Install-Module Microsoft.Graph -Scope CurrentUser`; consent at first `Connect-MgGraph` (admin consent once in Entra → Enterprise applications if required).

2.6 The in-scope data assets and control inventory

Description: The **data assets and Purview controls** the framework will assign ownership for — the in-scope SharePoint sites/Teams/domains, the sensitivity labels, DLP policies, retention policies, and governance domains — plus the **classification/sensitivity intent** that drives which are highest priority. Per Global Requirement 4. **Why:** You cannot assign an owner to an asset you have not enumerated; the

assessment script builds the inventory the RACI is applied against. **Verify:** Run `DGSTEWARD-Assess-Ownership.ps1` — it inventories labels, DLP/retention policies, and (via Graph, where available) governance domains, and flags which have no recorded owner/steward. **Implement:** Confirm the in-scope list with stakeholders before authoring the RACI.

2.7 Named stakeholders and the RACI authority

Description: Engagement with the **compliance, records, privacy, security, and legal** stakeholders and the business **data owners** who will be named in the RACI, plus the authority who approves it. Per Global Requirement 5. **Why:** A RACI is only real if the named Accountable/Responsible parties have agreed to it and an authority has ratified it; sensitive-role membership (insider risk, communication compliance, eDiscovery) must be confirmed by HR/privacy/legal. **Verify:** Confirm the named owners/stewards per domain and the approver of `Data-Owner/Steward RACI – Design`. **Implement:** Identify the owners and capture their agreement before configuring assignments. The *determination* of who holds a role is theirs; you implement it.

2.8 A pilot scope and review cadence agreed

Description: A **bounded pilot scope** (one or two governance domains / a small named asset set) to validate the framework on, and an **agreed review cadence** (e.g. quarterly re-certification, monthly gap chase — the interval is a business decision) with a change-control path. Per Global Requirement 6. **Why:** The framework is validated on a ring before estate-wide rollout, and the cadence must have an owner and an interval or it will not run. **Verify:** Confirm the named `Stewardship Pilot Domain – Stage1`, the cadence interval, and who owns running it. **Implement:** Define the pilot and agree the cadence and change control before expanding.

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — the solutions provide the technical configuration method and evidence, not the determination. Automation is **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

- **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires

(E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

- **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the

right plan.

- **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is

visible in the Purview portal.

- **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

- **Description:** Access with the least-privilege **Purview / compliance role** the solution names

(e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

- **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.
- **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.
- **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

- **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the

ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

- **Why required:** Configuration, automation, and the provided scripts run through these.
- **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.
- **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

- **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams,

endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.

- **Why required:** Purview policies key off the in-scope workloads and the classification scheme.
- **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with

stakeholders.

- **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

• **Description:** Engagement with the **compliance, records-management, privacy, security, and legal**

stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).

• **Why required:** Purview configuration must align to enterprise policy and legal/records requirements,

not technical defaults; several capabilities have HR/legal sensitivity.

• **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.

• **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

• **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate

on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.

• **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate

→ Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.

• **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change

window/approval path.

• **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

3. Implementation Guide — Data-Owner & Steward Framework (Microsoft Purview)

Follow these steps **in order**. Inventory and assessment come from the **Microsoft Purview portal** (purview.microsoft.com), **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph**. Role assignments are previewed with `-WhatIf` before `-Commit`. Artifact names follow the `<Purpose> – Purview – <Stage>` convention from the Overview and are reused verbatim.

> ***Model:*** *Assess ownership coverage → design the role model + RACI → configure assignments + cadence → validate on a **pilot domain** → confirm every gate → expand across the estate. Pattern: **Assess → Design → Configure → Pilot → Validate → Expand. Portal paths are cited as of authoring and the Purview portal — especially the Unified Catalog / Data Governance experience — is evolving; verify each path in your own tenant.** Capability availability is*

*licensing-dependent (Requirement 2.2). This framework assigns *accountability*; it does not author labels, DLP, or retention policies — those are piloted in simulation/test in their own solutions.*

Step 1 — Assess current ownership coverage (Assess)

1. Open the **Microsoft Purview portal** (`https://purview.microsoft.com`) and orient: confirm which

in-scope surfaces render (Information Protection → Labels, Data Loss Prevention → Policies, Data Lifecycle Management → Retention policies, Unified Catalog → Governance domains). Visibility tracks your licensing.

1. Run `DGSTEWARD-Assess-Ownership.ps1 -CsvPath .\OwnershipInventory.csv`. Read-only, it inventories the

in-scope Purview controls — sensitivity labels (`Get-Label`), DLP policies (`Get-DlpCompliancePolicy`), retention policies (`Get-RetentionCompliancePolicy`) — plus (via Graph, where the Data Governance API is available in your tenant) governance domains, and reports for each whether an **owner** and **steward** are recorded (in the object's comment/metadata or the domain roles). It writes `Data Ownership Inventory – Steward – Assess`.

1. Record the **coverage gap**: which controls/assets have no owner, no steward, or neither.

Pass/fail: you can state, per in-scope surface, a coverage number — "N of M labels have a recorded owner", "K DLP policies are unowned" — and a named list of gaps to close.

Step 2 — Design the role model and RACI (Design)

1. Using the coverage output and the role model in the Overview, define the **roles** and who fills them:

Data Owner, **Data Steward**, **Governance Domain Owner**, **Data Custodian / Control Owner**, and the strictly-separated sensitive roles (**Insider Risk**, **Communication Compliance**, **eDiscovery**).

1. Author the RACI `Data-Owner/Steward RACI – Design` as a CSV: one row per **governance activity**

(classify/label an asset, author/tune a DLP policy, author/tune a retention policy, re-certify ownership, review a disposition, respond to an access request, close an audit finding), with columns naming exactly one **Accountable** party and the **Responsible / Consulted / Informed** parties. Rule: **exactly one A per row**; every row has at least one R.

1. Map each in-scope control/asset from Step 1 to its **Data Owner** and **Data Steward** in a companion

assignment CSV (`Purview-Stewardship-Matrix.csv`) — the input the configuration script consumes.

1. Apply separation of duties: sensitive-role rows are **named, minimal, and separate** from general

stewardship, and confirmed with HR/privacy/legal. **Pass/fail:** every in-scope control/asset has a named Owner and Steward; every RACI activity has exactly one A; no sensitive role is over-assigned.

Step 3 — Preview and apply stewardship assignments (Configure, pilot-first)

1. Dry-run first: `DGSTEWARD-Set-Stewardship.ps1 -MatrixPath .\Purview-Stewardship-Matrix.csv -WhatIf`.

With `-WhatIf` (the default) it makes **no changes** — it prints exactly which member would be added to which role group, and which owner/steward tag would be written to which control's metadata, and flags any role group or object it cannot find.

1. Read every line against `Data-Owner/Steward RACI – Design`. Confirm no unintended broad grant and that

the sensitive-role rows match the HR/legal-approved list.

1. Apply to the **pilot domain first**: run with `-Commit` scoped to `Stewardship Pilot Domain – Stage1`.

Compliance-side assignments use `Add-RoleGroupMember`; control-ownership tags are written to the object's comment/metadata via the object's `Set-*` cmdlet (e.g. `Set-Label`); governance-domain owner/steward assignments are made in the Unified Catalog (portal, or Graph where the data-plane API is available — the script flags this and does not invent an API). Writes `Stewardship Assignments – Configure`.

1. **Pass/fail + rollback:** `DGSTEWARD-Test-Framework.ps1` reports every pilot control/asset now has a

recorded owner and steward. Rollback is `Remove-RoleGroupMember` / clearing the metadata tag — the script prints the exact reverse command for each change it makes.

Step 4 — Establish the review cadence (Configure)

1. Agree the cadence interval with the stakeholders (e.g. quarterly re-certification of owned assets,

monthly gap chase — the interval is a business decision, Requirement 2.8).

1. Record, per control/asset, a **last-reviewed** date and a **review-by** date (written into the object's

metadata tag alongside owner/steward, or tracked in the assignment CSV where the object has no writable metadata). This is what makes "overdue" computable.

1. Assign the cadence an **owner** (the Data Steward runs it; the Data Owner is Accountable). **Pass/fail:**

`DGSTEWARD-Check-Cadence.ps1` returns a due/overdue list and a named owner for running it.

Step 5 — Validate on the pilot domain (Pilot → Validate)

1. Run the full validation suite against `Stewardship Pilot Domain – Stage1` (see the Testing Guide): every

in-scope control/asset has an Owner and Steward, the RACI resolves to real identities, sensitive roles are separated, and the cadence produces a correct due/overdue list.

1. Produce `Accountability Coverage – Report` (`DGSTEWARD-Report-Accountability.ps1`) — the coverage

percentage and gap list for the pilot. **Pass/fail:** pilot coverage is 100% of in-scope controls, or every remaining gap is a named, owned action item.

Step 6 — Expand across the estate (Expand)

1. Broaden the assignment CSV from the pilot domain to the full in-scope set; re-run Step 3 (dry-run then

`-Commit`) domain by domain, not all at once.

1. Re-run the accountability report and confirm coverage across the estate; hand the running of the

cadence to the named stewards.

1. **Pass/fail:** estate coverage meets the agreed target; the cadence is scheduled and owned; every

sensitive role remains named, minimal, and separated.

> ***What is deferred (by design).*** **What* an asset's classification, retention, or records schedule is, *whether* a regulation applies, and *who may* hold a sensitive role are* ***determinations*** *made by compliance, records, privacy, and legal — not by this framework. The framework records and operates those decisions with named accountability and a cadence; it does not make them, and it does not author the underlying label/DLP/retention policies.*

5. Testing Guide — Data-Owner & Steward Framework (Microsoft Purview)

Run after building the ownership inventory and after each configuration step. Every test is safe (read-only or a `-WhatIf` preview). Validate on the `Stewardship Pilot Domain – Stage1` before rolling the framework across the estate.

Test 1 — Ownership inventory is complete

Do: Run `DGSTEWARD-Assess-Ownership.ps1 -CsvPath .\OwnershipInventory.csv`; compare its control count against what you see in the Purview portal (labels, DLP policies, retention policies, governance domains). **Expected:** Every in-scope control/asset appears in the inventory; each is marked *owned* / *steward- assigned* / *unowned* correctly, and the coverage number matches a manual spot-check.

Test 2 — RACI is well-formed

Do: Open `Data-Owner/Steward RACI – Design`; for each activity row count the A/R/C/I entries. **Expected:** Exactly **one Accountable** party per activity, at least one Responsible party per activity, and every named party resolves to a real person or group (no placeholders). No activity is orphaned.

Test 3 — Assignment dry-run makes no changes

Do: Run `DGSTEWARD-Set-Stewardship.ps1 -MatrixPath .\Purview-Stewardship-Matrix.csv -WhatIf`; afterward re-run `DGSTEWARD-Assess-Ownership.ps1`. **Expected:** The dry-run lists the intended owner/steward assignments and prints the reverse command for each; the inventory is unchanged — `-WhatIf` truly changed nothing.

Test 4 — Every pilot control/asset has an owner and a steward

Do: After `-Commit` on the pilot domain, run `DGSTEWARD-Test-Framework.ps1`. **Expected:** Every in-scope control/asset in `Stewardship Pilot Domain – Stage1` reports a recorded **Data Owner** and **Data Steward**; there are zero unowned in-scope items, or every remaining gap is a named, owned action.

Test 5 — Named roles resolve to real identities

Do: Run `DGSTEWARD-Report-Accountability.ps1`; it resolves each named owner/steward against Graph. **Expected:** Every name in the RACI and the assignment matrix resolves to a valid user or group; no assignment points at a departed or non-existent identity.

Test 6 — Separation of duties holds

Do: Review the sensitive-role rows (Insider Risk, Communication Compliance, eDiscovery) in the RACI and the corresponding role-group membership. **Expected:** These roles are named, minimal, and **separate** from general stewardship; the same person does not both steward general data and hold a sensitive role beyond the HR/legal-approved list.

Test 7 — Review cadence produces a correct due/overdue list

Do: Run `DGSTEWARD-Check-Cadence.ps1 -CadenceDays 90` (or your agreed interval). **Expected:** The script returns each control/asset's last-reviewed and review-by dates and correctly flags **due** and **overdue** items; an item just re-certified drops off the overdue list; an item past its review-by date appears on it.

Test 8 — Accountability report reflects reality

Do: Run `DGSTEWARD-Report-Accountability.ps1 -CsvPath .\Accountability.csv`; open the Purview portal and spot-check three controls. **Expected:** `Accountability Coverage – Report` shows the coverage percentage and gap list that match the portal; the three spot-checked controls' owner/steward match what the report claims.

Pass/fail gate

Gate	Requirement
Framework pass (pilot)	Tests 1–8 pass on `Stewardship Pilot Domain – Stage1`: inventory complete, RACI well-formed (one A per activity), assignments applied, every pilot item owned + steward-assigned, roles resolve, separation holds, cadence correct, report accurate
Expand pass	Tests 1, 3–5, 7–8 pass again after broadening from the pilot domain to the full estate, with coverage meeting the agreed target and the cadence scheduled and owned

6. Troubleshooting Guide — Data-Owner & Steward Framework (Microsoft Purview)

Field issues and how to resolve them, in the order you are likely to hit them. Every check here is safe (read-only or a `-WhatIf` preview).

A capability or surface is missing from the Purview portal

Symptom: A surface you expected to assign ownership for (records management, Audit Premium, Unified Catalog governance domains) does not appear, or shows "upgrade required." **Cause:** It is not licensed on the current plan — Purview is heavily plan-gated (E5 / E5 Compliance; the Unified Catalog / Data Governance side is consumption-billed via an Azure subscription; some features are in preview). **Fix:** Confirm the SKU in `Billing → Licenses` and the linked Azure subscription for Data Governance. Record the surface as **out-of-scope for ownership assignment for now** rather than forcing it. Verify availability against your own tenant and plan — the capability set evolves.

`Connect-IPPSSession` fails or cmdlets are "not recognized"

Symptom: Connection errors, repeated auth prompts, or `Get-Label` / `Get-DlpCompliancePolicy` not found. **Cause:** Missing/old ExchangeOnlineManagement module, or a basic-auth/legacy sign-in. **Fix:** `Install-Module ExchangeOnlineManagement -Scope CurrentUser -Force`; use a modern-auth/MFA admin account; run `Connect-IPPSSession` (Security & Compliance PowerShell) — the label/DLP/retention cmdlets live there, not in plain Exchange Online. Close and reopen PowerShell if a stale session is cached.

`DGSTEWARD-Assess-Ownership.ps1` shows everything as "Unowned"

Symptom: The inventory runs but every control reports no Owner/Steward. **Cause:** Ownership has not been applied yet, or it was recorded somewhere other than the object comment tag the scripts read (`Owner=…; Steward=…`). **Fix:** This is expected before Step 3. Author the assignment matrix and run `DGSTEWARD-Set-Stewardship.ps1` (dry-run, then `-Commit` on the pilot). The framework's convention is to store owner/steward in the object's **comment/metadata**; if your organization records ownership elsewhere (a CMDB, a separate register), keep the tag in sync so the scripts can evidence coverage.

The governance-domain rows are skipped

Symptom: `DGSTEWARD-Assess-Ownership.ps1` reports governance domains skipped. **Cause:** No `-DataGovernanceUri` was supplied, or the Unified Catalog data-plane API was not reachable — the Data Governance experience is evolving and partly in preview, and its API version/path is **not assumed** by the scripts. **Fix:** Assign and verify governance-domain **owners/stewards in the Purview portal** (Unified Catalog → the domain → its roles — verify the path in your tenant). Only supply `-DataGovernanceUri` if you have confirmed the current data-plane endpoint for your tenant; the script never invents it.

`DGSTEWARD-Set-Stewardship.ps1` "changed nothing"

Symptom: You ran the assignment script and no control was updated. **Cause:** This is by design — `-WhatIf` **is the default**. Without `-Commit` the script only previews and prints the reverse command for each intended change. **Fix:** Review the preview, confirm it matches `Data-Owner/Steward RACI – Design`, then re-run with `-Commit` — scoped to `Stewardship Pilot Domain – Stage1` first, then expand.

A role group named in the matrix "does not exist"

Symptom: `DGSTEWARD-Set-Stewardship.ps1` warns a role group was not found and skips the member add. **Cause:** The built-in role-group name differs in your tenant, the Purview portal has moved that role into the unified Purview RBAC model, or the role belongs to a capability not licensed. **Fix:** Verify the exact name in Purview portal → `Settings → Roles & scopes → Role groups` (verify path) and correct the matrix. Do not assume a name — the portal is evolving. The tag write still succeeds; only the optional role-group add is skipped.

`Set-Label` / `Set-DlpCompliancePolicy` reports access denied on `-Commit`

Symptom: The commit errors when writing the ownership tag to a control. **Cause:** The account lacks the least-privilege role to edit that object (e.g. Information Protection Admin for labels, Compliance Administrator for DLP/retention). **Fix:** Run the commit as an account in the correct built-in role group (Requirement 2.3). Editing a control's comment is a low-impact metadata change, but it still requires the managing role.

Named owners/stewards don't resolve in the accountability report

Symptom: `DGSTEWARD-Report-Accountability.ps1 -ResolveIdentities` flags `OwnerResolves = False`. **Cause:** The name is a typo, a display-name that doesn't match the directory, or the person has left — or the Graph read scopes (`User.Read.All`, `Group.Read.All`) weren't consented. **Fix:** Prefer **UPNs or security-group names** in the matrix over free-text display names; consent the read scopes at `Connect-MgGraph` (admin consent once in Entra → Enterprise applications if required); re-assign any control whose owner/steward has departed. A stale assignment is exactly the gap this report exists to surface.

Cadence check shows everything "NEVER-REVIEWED"

Symptom: `DGSTEWARD-Check-Cadence.ps1` reports no review dates. **Cause:** The `Reviewed=/ReviewBy=` dates were never written — assignments were made without recording a review date. **Fix:** When you assign ownership, include `Reviewed` and `ReviewBy` in the matrix so the tag carries them; re-run `DGSTEWARD-Set-Stewardship.ps1 -Commit`. Going forward, each re-certification updates `Reviewed=` (and the script derives `ReviewBy = Reviewed + CadenceDays` if you don't set it explicitly).

"Who is allowed to hold the Insider Risk / eDiscovery role?"

Symptom: Uncertainty over assigning a sensitive role in the RACI. **Cause:** These roles see sensitive employee content; membership is an HR/privacy/legal **determination**, not a technical default. **Fix:** Leave

the sensitive-role rows to the HR/privacy/legal-approved list; keep them named, minimal, and **separate** from general stewardship. The framework implements the assignment they approve — it does not decide who may hold the role.

The framework "passed" but ownership drifted a quarter later

Symptom: Coverage was 100% at rollout; now controls are unowned or overdue again. **Cause:** The **review cadence** isn't running or has no owner — a framework without a cadence decays. **Fix:** Schedule `DGSTEWARD-Check-Cadence.ps1` on the agreed interval, give the cadence a named steward to run it and a Data Owner accountable for it running, and treat the overdue list as work, not information. This is the whole point of the cadence deliverable.

7. Best Practices — Data-Owner & Steward Framework (Microsoft Purview)

- **Exactly one Accountable party per activity — no exceptions.** The single most common failure of a

RACI is two people sharing accountability, which is the same as nobody owning it. Every activity row names exactly one A; if you cannot name one, the activity has no owner and the gap is real.

- **Separate the Data Owner from the Data Custodian.** The business **Data Owner** decides intent

(classification, retention, access); the IT/compliance **Custodian / Control Owner** operates the control (the label, the DLP policy, the retention policy). Conflating them is how "IT owns compliance" — a known anti-pattern — creeps back in. Name both.

- **Assign from least privilege, not from Global Admin.** Add stewards to the specific built-in role group

or governance-domain role their function needs; reserve Global Administrator for the rare tenant-wide action. Broad admin "to get unblocked" undermines the accountability you are building.

- **Keep the sensitive roles named, minimal, and separate.** Insider Risk, Communication Compliance, and

eDiscovery see sensitive employee content — their assignments are minimal, named, separate from general stewardship, and confirmed with HR/privacy/legal. *Who* may hold them is their determination, not a technical default.

- **Always dry-run assignments.** Run `DGSTEWARD-Set-Stewardship.ps1 -WhatIf` and read every line before

`-Commit`; the script prints the exact reverse command for each change so rollback is trivial.

- **Pilot on a bounded domain before the estate.** Prove the RACI, the assignments, and the cadence on one

or two governance domains / a small asset set first. A framework is cheap to fix on a ring and expensive to fix estate-wide.

- **Ownership without a cadence decays — schedule it and give it an owner.** Re-certify owned assets on

the agreed interval, chase unowned/overdue items every cycle, and produce the accountability report. The Data Steward runs the cadence; the Data Owner is Accountable for it running.

- **Record a last-reviewed and review-by date on every owned item.** "Overdue" is only computable if the

dates exist. Write them into the object's metadata where it is writable, or track them in the assignment matrix where it is not.

- **Own the controls, not just the data.** A label, a DLP policy, a retention policy, and a governance

domain each need an owner and a steward, the same as a data asset. An unowned control drifts silently.

- **Treat portal paths and the Data Governance API as provisional.** The Purview portal — especially the

Unified Catalog / Data Governance experience — is actively evolving and partly in preview; verify each path and API version in your own tenant rather than trusting a screenshot or a version from last quarter. Do not assume a cmdlet or API that you have not confirmed exists.

- **The framework records decisions; it does not make them.** *What* an asset is classified as, *what* its

retention/records schedule is, *whether* a regulation applies, and *who may* hold a sensitive role are determinations for compliance, records, privacy, and legal. Assign accountability for operating those decisions; defer the decisions themselves.

8. Deliverables Summary — Data-Owner & Steward Framework (Microsoft Purview)

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	The accountability operating model — roles (Data Owner, Data Steward, Governance Domain Owner, Custodian), the RACI, the review cadence, least-privilege, limitations, and the naming convention
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Purview)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered portal + S&C PowerShell + Graph steps (assess coverage, design RACI, apply assignments pilot-first, establish cadence, validate, expand), each control/asset named, pass/fail gates + rollback
5. Testing Guide	Validation procedures with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

PowerShell scripts

Script	What it does
`DGSTEWARD-Assess-Ownership.ps1`	Read-only ownership/coverage discovery: inventories in-scope Purview controls — sensitivity labels, DLP policies, retention policies via S&C PowerShell, and (via Graph, where available) governance domains — and reports for each whether an owner/steward is recorded. Writes `Data Ownership Inventory – Steward – Assess`
`DGSTEWARD-Set-Stewardship.ps1`	Stewardship-assignment helper: from a matrix CSV, adds named members to built-in role groups and writes owner/steward + review-date tags to control metadata; `-WhatIf` **by default**, `-Commit` to apply; prints the reverse command for each change. Writes `Stewardship Assignments – Configure`
`DGSTEWARD-Test-Framework.ps1`	Read-only validation: confirms every in-scope control/asset has a recorded Data Owner and Data Steward, that named parties resolve, and that separation-of-duties holds; reports any gap
`DGSTEWARD-Report-Accountability.ps1`	Read-only accountability/coverage report: coverage percentage, gap list, and identity resolution of every named owner/steward against Graph. Writes `Accountability Coverage – Report`
`DGSTEWARD-Check-Cadence.ps1`	Read-only review-cadence health helper: computes each control/asset's last-reviewed / review-by dates and flags due and overdue items against the agreed interval. Writes `Review Cadence – Health`

All scripts are **safe by default** — read-only, or a `-WhatIf` preview requiring an explicit `-Commit` to change anything. No credentials or secrets are stored in code; every script uses interactive modern-auth (`Connect-IPPSSession` / `Connect-MgGraph`). Scripts do not invent cmdlets or Data Governance API versions — where the Unified Catalog data-plane API is not confirmed available in your tenant, the governance-domain steps are flagged for portal action rather than assumed.

What the buyer receives

- This implementation guide as a **Word** document and **PDF**.
- All five **PowerShell scripts** in a single ZIP.
- A repeatable process following the model: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand.**

Outcome

A Microsoft Purview data-owner/steward operating framework in place — every in-scope data asset and governance control assigned a named **Data Owner** and **Data Steward**, responsibility recorded in a well-formed **RACI** (one Accountable party per activity), and the whole thing on a **review cadence** so accountability stays current — configured, piloted on a bounded domain, validated against pass/fail gates, and defensible. Regulatory, records, and legal determinations remain with the organization's compliance, records, privacy, and legal functions; this framework operates them with accountability, it does not make them.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

DGSTEWARD-Assess-Ownership.ps1

Purpose: Builds the ownership-coverage baseline for the data-owner/steward framework: inventories the in-scope Purview controls (sensitivity labels, DLP policies, retention policies) via Security & Compliance PowerShell, and — where the Data Governance data-plane is available in your tenant — governance domains via Microsoft Graph, reporting for each whether an owner and steward are recorded in its comment/metadata. Read-only — makes no changes. Writes the export "Data Ownership Inventory - Steward - Assess". HOW TO RUN powershell -ExecutionPolicy Bypass -File .\DGSTEWARD-Assess-Ownership.ps1 -CsvPath .\OwnershipInventory.csv PowerShell 5.1+ or 7+. Requires ExchangeOnlineManagement (Connect-IPPSSession); optional Microsoft Graph SDK for governance domains. Graph read scopes (optional): Organization.Read.All. EXPECTED RESULTS A console + optional CSV inventory of each control with Owner/Steward = recorded or (unowned). PASS = you can state a coverage number per surface and a named list of gaps to close. .NOTES Purview — Data-Owner & Steward Framework. Read-only. No secrets in code. Owner/steward convention: a tag written to the object comment as "Owner=<name>; Steward=<name>; Reviewed=<date>; ReviewBy=<date>". The Unified Catalog / Data Governance experience is evolving and partly in preview — the governance-domain read is attempted only if you supply -DataGovernanceUri and is flagged, never assumed. Verify in your tenant. > [CmdletBinding()] param([string]$CsvPath, [string]$DataGovernanceUri, # optional Graph data-governance endpoint if available in your tenant; skipped if omitted [string]$ReportName = 'Data Ownership Inventory - Steward - Assess')

How to run: powershell -ExecutionPolicy Bypass -File .\DGSTEWARD-Assess-Ownership.ps1 -CsvPath .\OwnershipInventory.csv PowerShell 5.1+ or 7+. Requires ExchangeOnlineManagement (Connect-IPPSSession); optional Microsoft Graph SDK for governance domains. Graph read scopes (optional): Organization.Read.All. EXPECTED RESULTS A console + optional CSV inventory of each control with Owner/Steward = recorded or (unowned). PASS = you can state a coverage number per surface and a named list of gaps to close. .NOTES Purview — Data-Owner & Steward Framework. Read-only. No secrets in code. Owner/steward convention: a tag written to the object comment as "Owner=<name>; Steward=<name>; Reviewed=<date>; ReviewBy=<date>". The Unified Catalog / Data Governance experience is evolving and partly in preview — the governance-domain read is attempted only if you supply -DataGovernanceUri and is flagged, never assumed. Verify in your tenant. > [CmdletBinding()] param([string]$CsvPath, [string]$DataGovernanceUri, # optional Graph data-governance endpoint if available in your tenant; skipped if omitted [string]$ReportName = 'Data Ownership Inventory - Steward - Assess')

DGSTEWARD-Check-Cadence.ps1

Purpose: Runs the review-cadence check that keeps the framework from decaying: reads each in-scope control's Reviewed= (last-reviewed) and ReviewBy= dates from its ownership tag, computes days-until-due against the agreed interval, and flags items that are DUE-SOON or OVERDUE for the steward to re-certify. Read-only — makes no changes. Writes the export "Review Cadence - Health". HOW TO RUN powershell -ExecutionPolicy Bypass -File .\DGSTEWARD-Check-Cadence.ps1 -CadenceDays 90 -WarnWithinDays 14 PowerShell 5.1+/7+. Requires ExchangeOnlineManagement (Connect-IPPSSession). EXPECTED RESULTS A due/overdue table. PASS = zero overdue items (or

every overdue item is a named, owned action). An item just re-certified drops off the overdue list; an item past ReviewBy appears on it. .NOTES Purview — Data-Owner & Steward Framework. Read-only. No secrets in code. If a control has no ReviewBy= date, ReviewBy is derived as Reviewed + CadenceDays; if it has neither, it is reported as NEVER-REVIEWED. > [CmdletBinding()] param([int]$CadenceDays = 90, [int]$WarnWithinDays = 14, [string]$CsvPath, [string]$ReportName = 'Review Cadence - Health')

How to run: powershell -ExecutionPolicy Bypass -File .\DGSTEWARD-Check-Cadence.ps1 -CadenceDays 90 -WarnWithinDays 14 PowerShell 5.1+/7+. Requires ExchangeOnlineManagement (Connect-IPPSSession). EXPECTED RESULTS A due/overdue table. PASS = zero overdue items (or every overdue item is a named, owned action). An item just re-certified drops off the overdue list; an item past ReviewBy appears on it. .NOTES Purview — Data-Owner & Steward Framework. Read-only. No secrets in code. If a control has no ReviewBy= date, ReviewBy is derived as Reviewed + CadenceDays; if it has neither, it is reported as NEVER-REVIEWED. > [CmdletBinding()] param([int]$CadenceDays = 90, [int]$WarnWithinDays = 14, [string]$CsvPath, [string]$ReportName = 'Review Cadence - Health')

DGSTEWARD-Report-Accountability.ps1

Purpose: Produces the accountability coverage report: the percentage of in-scope Purview controls with a recorded Owner and Steward, the gap list, and — where the Microsoft Graph SDK is available — resolution of each named owner/steward to a real user or group so stale assignments (departed people, typo'd names) surface. Read-only — makes no changes. Writes the export "Accountability Coverage - Report". HOW TO RUN powershell -ExecutionPolicy Bypass -File .\DGSTEWARD-Report-Accountability.ps1 -CsvPath .\Accountability.csv PowerShell 5.1+/7+. Requires ExchangeOnlineManagement (Connect-IPPSSession). Optional Microsoft Graph SDK for identity resolution; Graph read scopes: User.Read.All, Group.Read.All. EXPECTED RESULTS A coverage percentage, a gap list, and (if Graph is used) an "identity resolves: yes/no" column. PASS = coverage meets the agreed target and every named party resolves. .NOTES Purview — Data-Owner & Steward Framework. Read-only. No secrets in code. > [CmdletBinding()] param([string]$CsvPath, [switch]$ResolveIdentities, [string]$ReportName = 'Accountability Coverage - Report')

How to run: powershell -ExecutionPolicy Bypass -File .\DGSTEWARD-Report-Accountability.ps1 -CsvPath .\Accountability.csv PowerShell 5.1+/7+. Requires ExchangeOnlineManagement (Connect-IPPSSession). Optional Microsoft Graph SDK for identity resolution; Graph read scopes: User.Read.All, Group.Read.All. EXPECTED RESULTS A coverage percentage, a gap list, and (if Graph is used) an "identity resolves: yes/no" column. PASS = coverage meets the agreed target and every named party resolves. .NOTES Purview — Data-Owner & Steward Framework. Read-only. No secrets in code. > [CmdletBinding()] param([string]$CsvPath, [switch]$ResolveIdentities, [string]$ReportName = 'Accountability Coverage - Report')

DGSTEWARD-Set-Stewardship.ps1

Purpose: Applies the stewardship assignment matrix: writes an "Owner=; Steward=; Reviewed=; ReviewBy=" tag to each in-scope control's comment/metadata (sensitivity label, DLP policy, retention policy) and, where named, adds a member to the built-in role group that steward needs. SAFE BY DEFAULT — runs as a -WhatIf preview that changes nothing and prints the exact reverse command for each action; pass -Commit to apply. Writes the record "Stewardship Assignments - Configure". HOW TO RUN Preview (default, no changes): powershell -ExecutionPolicy Bypass -File .\DGSTEWARD-Set-Stewardship.ps1 -MatrixPath .\Purview-Stewardship-Matrix.csv -WhatIf Apply (pilot domain first): powershell -ExecutionPolicy Bypass -File .\DGSTEWARD-Set-Stewardship.ps1 -MatrixPath

.\Purview-Stewardship-Matrix.csv -Commit CSV columns: ControlType,Name,Identity,Owner,Steward,RoleGroup,Member,Reviewed,ReviewBy PowerShell 5.1+/7+. Requires ExchangeOnlineManagement (Connect-IPPSSession). EXPECTED RESULTS Preview lists every intended tag write and role-group add with its reverse command; -Commit applies them. PASS = re-running DGSTEWARD-Assess-Ownership.ps1 shows the controls now Owned. .NOTES Purview — Data-Owner & Steward Framework. Safe by default (-WhatIf). No secrets in code. Governance-domain owner/steward assignment is made in the Unified Catalog portal (or a confirmed Graph data-plane API) — this script does not invent that API; it flags GovernanceDomain rows for portal action. > [CmdletBinding(SupportsShouldProcess=$true, ConfirmImpact='High')] param([Parameter(Mandatory=$true)][string]$MatrixPath, [switch]$Commit, [string]$ReportName = 'Stewardship Assignments - Configure')

How to run: Preview (default, no changes): powershell -ExecutionPolicy Bypass -File .\DGSTEWARD-Set-Stewardship.ps1 -MatrixPath .\Purview-Stewardship-Matrix.csv -WhatIf Apply (pilot domain first): powershell -ExecutionPolicy Bypass -File .\DGSTEWARD-Set-Stewardship.ps1 -MatrixPath .\Purview-Stewardship-Matrix.csv -Commit CSV columns: ControlType,Name,Identity,Owner,Steward,RoleGroup,Member,Reviewed,ReviewBy PowerShell 5.1+/7+. Requires ExchangeOnlineManagement (Connect-IPPSSession). EXPECTED RESULTS Preview lists every intended tag write and role-group add with its reverse command; -Commit applies them. PASS = re-running DGSTEWARD-Assess-Ownership.ps1 shows the controls now Owned. .NOTES Purview — Data-Owner & Steward Framework. Safe by default (-WhatIf). No secrets in code. Governance-domain owner/steward assignment is made in the Unified Catalog portal (or a confirmed Graph data-plane API) — this script does not invent that API; it flags GovernanceDomain rows for portal action. > [CmdletBinding(SupportsShouldProcess=$true, ConfirmImpact='High')] param([Parameter(Mandatory=$true)][string]$MatrixPath, [switch]$Commit, [string]$ReportName = 'Stewardship Assignments - Configure')

DGSTEWARD-Test-Framework.ps1

Purpose: The pass/fail gate for the framework: re-reads the in-scope Purview controls (sensitivity labels, DLP policies, retention policies) via Security & Compliance PowerShell, confirms each carries an Owner and a Steward tag, and reports any gap. Optionally cross-checks named owners/stewards against an expected assignment CSV. Read-only — makes no changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\DGSTEWARD-Test-Framework.ps1 With expected matrix: powershell -ExecutionPolicy Bypass -File .\DGSTEWARD-Test-Framework.ps1 -MatrixPath .\Purview-Stewardship-Matrix.csv PowerShell 5.1+/7+. Requires ExchangeOnlineManagement (Connect-IPPSSession). EXPECTED RESULTS A per-control PASS/FAIL table and an overall gate result. PASS = zero unowned in-scope controls (or every gap is a named, expected exception). .NOTES Purview — Data-Owner & Steward Framework. Read-only. No secrets in code. Owner/steward are read from the object comment tag written by DGSTEWARD-Set-Stewardship.ps1. > [CmdletBinding()] param([string]$MatrixPath, [string]$CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\DGSTEWARD-Test-Framework.ps1 With expected matrix: powershell -ExecutionPolicy Bypass -File .\DGSTEWARD-Test-Framework.ps1 -MatrixPath .\Purview-Stewardship-Matrix.csv PowerShell 5.1+/7+. Requires ExchangeOnlineManagement (Connect-IPPSSession). EXPECTED RESULTS A per-control PASS/FAIL table and an overall gate result. PASS = zero unowned in-scope controls (or every gap is a named, expected exception). .NOTES Purview — Data-Owner & Steward Framework. Read-only. No secrets in code. Owner/steward are read from the object comment tag written by DGSTEWARD-Set-Stewardship.ps1. > [CmdletBinding()] param([string]$MatrixPath, [string]$CsvPath)